

1. 项目定位

AptInsight 是面向公寓管理系统运营人员的智能数据分析助手。

它不是普通客服，也不是替管理员操作页面的按钮助手，而是基于现有业务数据库提供自然语言数据分析能力：

管理员输入经营问题

- > Agent 理解问题
- > 生成只读 SQL
- > 查询 MySQL 业务库
- > 返回表格、图表和运营总结

一句话定义：

AptInsight 是公寓管理系统后台的智能运营分析助手，支持管理员通过自然语言查询业务数据，自动完成 SQL 生成、只读查询、数据可视化和经营分析总结。

2. 为什么要做这个模块

现有后台系统已经可以完成公寓、房间、预约、租约、用户、岗位等基础管理，但传统后台页面通常只能查询固定条件下的列表数据。

运营人员经常会有临时分析需求：

问题类型	传统后台不足	AptInsight 价值
跨表统计	页面只展示单表或固定联表结果	自动关联公寓、房间、预约、租约
趋势对比	需要手动导出后再统计	自动按日、按月聚合并生成图表
经营诊断	页面只给数据，不给结论	输出业务总结和风险提示
临时问题	每个问题都要开发新接口	自然语言转 SQL 即时查询

3. 当前系统基础

当前项目已经具备 AptInsight 需要的业务数据基础。

模块	已有能力
公寓管理	公寓信息、地区、发布状态、配套、标签、杂费
房间管理	房间信息、租金、所属公寓、发布状态、属性、租期、支付方式
预约管理	看房预约、预约状态、预约时间、预约用户
租约管理	租约、租金、押金、租期、支付方式、签约状态、来源
用户管理	移动端用户、后台员工、岗位、账号状态
行为数据	浏览历史，记录用户浏览房间的时间

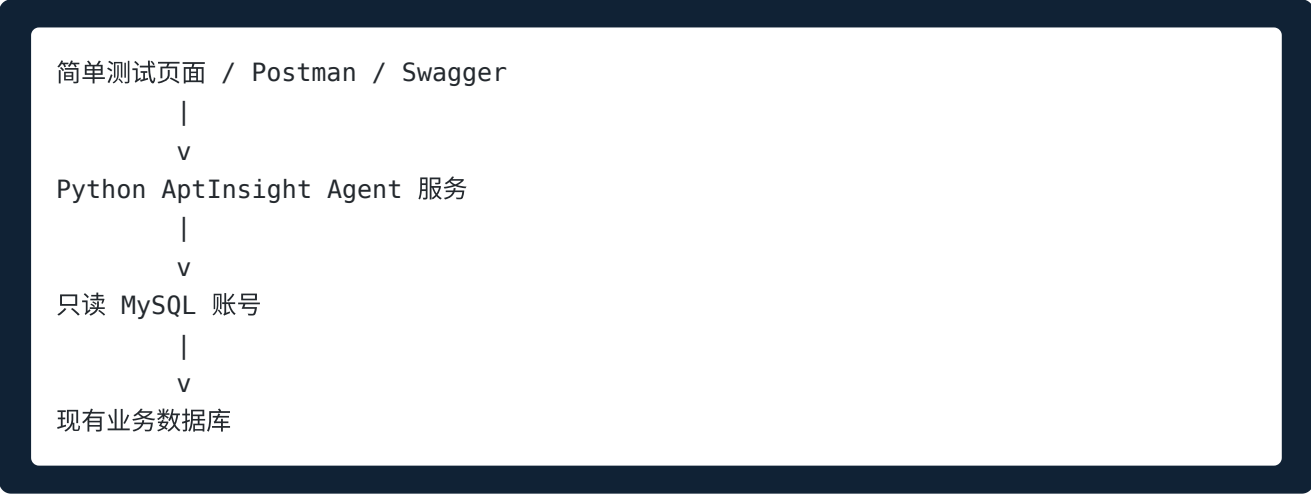
这些数据足够支撑第一版运营分析，包括预约趋势、租约统计、房源结构、租金分布、浏览热度和基础经营诊断。

4. 阶段规划

4.1 第一阶段：独立助手

目标：先验证 Agent 能否准确分析现有数据库，不改动当前前后端主流程。

形态：



第一阶段重点：

1. 整理数据库字典和指标口径。
2. 实现自然语言转 SQL。
3. 实现 SQL 安全校验。
4. 实现查询结果表格化。
5. 实现图表配置生成。
6. 实现运营总结生成。

7. 用真实业务表做测试集。

APTINSIGHT

4.2 第二阶段：集成后台

目标：将验证通过的 Agent 集成到现有管理后台。

形态：

```
Vue3 后台页面
  |
  v
Spring Boot /admin/ai/chat
  |
  v
Python AptInsight Agent 服务
  |
  v
MySQL 业务数据库
```

第二阶段重点：

1. 后台新增菜单：智能运营分析。
2. Spring Boot 新增 AI 转发接口。
3. 复用当前 JWT 登录鉴权。
4. 前端展示对话、SQL、表格、ECharts 图表。
5. 保存查询日志和错误信息。

4.3 第三阶段：运营增强

目标：从“问答式分析”升级为“主动运营辅助”。

可扩展能力：

1. 运营日报自动生成。
2. 空置风险房间识别。
3. 预约高但签约低的公寓诊断。
4. 多轮追问和上下文继承。
5. 常用指标看板沉淀。
6. 复杂报表导出。

5. 第一版 MVP 范围

第一版只做高价值、低风险能力。

能力	是否纳入 MVP	说明
自然语言提问	是	管理员输入中文问题
Text-to-SQL	是	只生成 SELECT 查询
SQL 安全校验	是	必须做，不能信任模型输出
MySQL 查询	是	使用只读账号
表格结果	是	所有查询至少返回表格
图表结果	是	柱状图、折线图、饼图、表格
AI 业务总结	是	根据查询结果输出简洁分析
查询日志	建议	第一版可以先记录到本地或数据库
多轮追问	暂缓	第二版再做
主动日报	暂缓	第三版再做
自动执行写操作	不做	永远不允许增删改业务数据

6. 核心分析场景

6.1 预约分析

典型问题：

本月各公寓预约量排名
最近 30 天看房预约趋势
哪个区的预约量最高
待看房、已看房、已取消分别有多少

当前 schema 中 `view_appointment` 只有 `apartment_id`，没有 `room_id`，因此第一版只能做公寓维度预约分析，不能直接做房间维度预约分析。

6.2 租约分析

典型问题：

本月新增租约数量
最近 6 个月租约增长趋势
当前租约状态占比
各公寓已签约租约数量排名

APTINSIGHT

租约表 `lease_agreement` 中有 `apartment_id` 和 `room_id`，可以按公寓、房间、租约状态、签约时间等维度统计。

6.3 房源分析

典型问题：

各公寓房间数量
各区域已发布房间数量
不同租金区间的房间数量
哪些已发布房间当前没有有效租约

房源分析主要依赖 `room_info`、`apartment_info`、`lease_agreement`。

6.4 租金分析

典型问题：

各公寓平均租金
各区域平均租金
租金最高的 10 个房间
当前已签约合同的月租金合计

注意：当前数据库没有账单表、支付流水表，因此第一版只能分析“合同租金”或“月租金规模”，不能称为真实已收款收入。

6.5 浏览热度分析

典型问题：

最近 30 天浏览量最高的房间
浏览量高但没有签约的房间
哪些房间长期没有浏览记录

浏览分析主要依赖 `browsing_history` 和 `room_info`。

6.6 经营诊断

典型问题：

- 哪些公寓预约量高但签约量低
- 哪些已发布房间存在空置风险
- 帮我总结当前公寓运营情况
- 哪个区域供给多但需求弱

经营诊断不是只返回 SQL 结果，还需要给出业务解释、可能原因和建议动作。

7. 明确不做的事情

第一版必须控制边界，避免做成不可控的大模型系统。

不做内容	原因
不执行 INSERT/UPDATE/DELETE	防止破坏业务数据
不让大模型直接连接数据库	必须经过 SQL 校验和只读账号
不返回身份证号、密码等敏感字段	避免隐私泄露
不做真实收入确认	当前没有支付流水
不声称能分析房间预约量	当前预约表没有房间字段
不依赖前端直连 Agent	集成阶段应通过 Spring Boot 保持鉴权和统一返回

8. 成功标准

第一阶段成功标准：

- 常见 20 条运营问题中，至少 16 条能生成正确 SQL 或给出合理拒答。
- 所有 SQL 均为只读查询，不出现增删改、DDL、多语句。
- 查询默认带 `is_deleted = 0` 过滤。
- 查询结果能返回结构化列、行、图表类型和总结。
- 对 schema 不支持的问题能明确说明缺少数据，而不是编造结果。

第二阶段成功标准：

- 管理员登录后可在后台菜单打开“智能运营分析”页面。

2. 页面可以提问、查看 SQL、表格、图表和总结。
3. Spring Boot 负责鉴权和转发，Python Agent 不直接暴露给未登录用户。
4. Agent 调用失败时前端有清晰错误提示。